SOUTH LAKE HOME EDUCATORS



Lesson 13

A Hit That Reacts

Semester 1, Week 13 - Basic Animation

At a Glance

Lesson 13 objectives & materials

By the end of today, students will:

- Sequence a flash before finalizing an existing state change
- Spawn a fully independent, self-cleaning explosion effect
- Compare how 3 engine families pause mid-script (or don't)
- Confirm game logic timing is completely unchanged

What We're Using Today

- Demo laptop with all 8 engines verified
- Scratch, for the live lecture demo
- One new asset: explosion_placeholder.png
- Unreal reuses its existing Cube/Sphere shapes
- Your own [YourName]Game project from Lesson 12

IGDA framework tie-in: Visual Art - basic animation, juice/feedback fundamentals (introductory); Programming - timed/asynchronous execution (intermediate).

Today's Agenda



0:00 - 0:05	Welcome back, Lesson 12 recap
0:05 - 0:15	A Hit That Reacts: Two New Visual Touches
0:15 - 0:40	Pausing Mid-Script: Three Engine Families, Three Solutions
0:40 - 0:50	An Explosion That Minds Its Own Business
0:50 - 0:55	Introduce lab exercise, hand out engine-specific handouts
LAB TIME	***— A Hit That Reacts —***
0:55 - 1:50	Wire up the flash and explosion, verify game logic unchanged
1:50 - 2:00	Exit check: best explosion wins bragging rights



Welcome Back

Your game has a voice. Now it gets a body.

5 min

Welcome Back

5 min

Quick recap of Lesson 12:

- Four sounds wired into their exact matching events
- Since Lesson 4, a hit just vanishes - functional, not satisfying
- Today: a brief flash, then a small explosion, before it disappears

Open your own project from last week today, not a new one. One new image is provided (explosion_placeholder.png) - Unreal students reuse existing shapes, TIC-80 students draw their own.



A Hit That Reacts

Two new visual touches - and one strict rule.

10 min

Two New Visual Touches

10 min

First: the enemy itself briefly flashes - a reaction ON the thing that got hit. Second: a completely separate explosion effect appears at that spot and plays itself out - independent of the enemy entirely.

The rule for today: score, lives, wave, and win/lose logic don't change WHEN they happen. Only how the enemy's removal LOOKS is changing.

If you find yourself moving scoring code around this week, stop and re-check the Handout.



Pausing Mid-Script

Three engine families, three genuinely different solutions.

25 min

Three Solutions, One Problem

25 min

To see a flash, the enemy must stick around a moment after being hit - something in our code needs to pause, without freezing the whole game.

Godot & Unity	await / Coroutine - the language itself can pause mid-function.
GameMaker & TIC-80	No blocking wait - split into "trigger" and "finish" via an Alarm or a countdown.
Unreal & 001GC	Delay node pauses a Blueprint wire; 001GC's Flash manages its own timing.



Minds Its Own Business

One object, one job, cleans up after itself.

10 min

An Explosion That Minds Its Own Business

10 min

The explosion spawns at the hit location and immediately forgets everything else - no score, no wave, not even which enemy made it. It plays itself out and cleans itself up, on its own.

This is the exact same discipline your bullets have followed since Lesson 3, and your enemies since Lesson 5. Every object, one job, cleans up after itself. The explosion is just the newest member of that club.



Today's Exercise & Lab

A Hit That Reacts - building the feedback loop.

5 min

A Hit That Reacts

5 min

Add a flash and an independent explosion effect - built on top of your existing project, with zero change to game-state timing.

Level 1 - Minimal

Every hit flashes the enemy and spawns an explosion effect.

MVP - a complete finish for today.

Level 2 - Complete

Correct on every hit, no stuck enemies, timing unaffected.

Everything works end to end.

Level 3 - Refined

Smoother fades, a scaling explosion, or a multi-frame flicker.

Ready to share.



Test a FULL round when you're done - shooting, a wave clear, even a loss - and confirm nothing about your SCORING changed, only the visuals.

Lab Time Notes

- Watch for score/win-check logic accidentally moved along with the delay - it should stay immediate; only the enemy's final removal is delayed.
- GameMaker and TIC-80 need extra circulating time this week - splitting logic across two events/table fields is a bigger structural change than usual.
- 001 Game Creator: the enemy's own flash may be brief or inconsistent - that's expected. The explosion effect is the reliable part to check first.
- This is a fun, visual week - let students actually watch a few hits happen slowly, and consider a quick show-and-tell of favorite explosion tweaks.

It Reacts!

A flash, a burst, then gone - getting hit finally looks like getting hit.

Next week: difficulty scaling across waves, and optional stretch features.

Keep playtesting a full round at home if you'd like - try a Level 3 animation idea if you haven't already.